

PRODUCT DOCUMENTATION

CodeReview AI

프로젝트 상세 PRD · 기능명세서 · API명세서

AI 기반 코드 리뷰 · 오류 수정 · 실행 · 챗봇 · 협업 하이브리드 리뷰 플랫폼

문서 버전 1.0 | 작성일 2026-07-30 | 상태: Draft for Web Implementation

항목	내용
문서 구분	PRD + 기능명세서 + API명세서 (통합 산출물)
제품명	CodeReview AI
현재 구현	단일 HTML SPA (Vanilla JS, 약 2,583 LOC)
목표	웹 애플리케이션으로 구조화·확장 구현
외부 연동	Groq Chat Completions API, Piston Execute API, GitHub Raw
저장소	sessionStorage (API Key), localStorage (채팅·협업)

목차

PART A. 프로젝트 상세 PRD (Product Requirements Document)

1. 제품 개요 및 비전
2. 문제 정의 및 기획
3. 목표 사용자 및 페르소나
4. 제품 목표 / 비목표 (In / Out of Scope)
5. 성공 지표 (KPI)
6. 시스템 아키텍처 개요
7. 사용자 여정 (User Journey)
8. 정보 구조 및 화면 구성
9. 비기능 요구사항
10. 제약사항 · 리스크 · 로드맵

PART B. 기능명세서 (Functional Specification)

1. 기능 목록 총괄
2. 모듈별 상세 기능 명세
3. 데이터 모델 (클라이언트)
4. 권한 · 상태 전이
5. 예외 및 오류 처리

PART C. API명세서 (API Specification)

1. 외부 API 연동 개요
2. Groq Chat Completions
3. Piston Code Execution
4. GitHub Raw Content
5. 프롬프트 계약 (JSON Schema)
6. 향후 백엔드 API 설계 초안

부록. 용어집 · 참고

PART A

프로젝트 상세 PRD (Product Requirements Document)

1. 제품 개요 및 비전

CodeReview AI는 개발자가 작성한 소스 코드를 브라우저에서 즉시 분석·수정·실행하고, 대화형 어시스턴트와 팀 협업(PR 기반 AI+사람 하이브리드 리뷰)까지 한 화면에서 수행하는 AI 네이티브 코드 품질 플랫폼이다.

현재 산출물은 Vanilla JS 단일 HTML 프로토타입이며, 본 문서는 이를 정식 웹 서비스로 확장·재구현하기 위한 제품 요구사항의 기준 문서(Source of Truth)로 사용한다.

1.1 제품 한 줄 정의

「코드를 붙이면, AI가 리뷰·고치고·돌리고·설명하고, 팀과 PR로 합의한다.」

1.2 핵심 가치 제안 (Value Proposition)

- 즉시성: 설치 없이 브라우저에서 리뷰·수정·실행 완료
- 다기능 통합: 리뷰 / 오류수정 / 실행 / 챗봇 / 협업을 한 제품에 통합
- 하이브리드 리뷰: AI 1차 분석 + 사람 최종 승인으로 품질·책임 균형
- 학습 친화: 점수·등급·샘플 코드·적용 버튼으로 초급자도 개선 루프 학습
- 비용 효율: Groq 무료 티어 수준의 LLM + 공개 샌드박스 실행(Piston) 활용

2. 문제 정의 및 기회

2.1 현재 문제

문제	영향	기존 대안의 한계
코드 리뷰 대기 시간 김	피드백 루프 지연, 학습 속도 저하	동료 리뷰·CI 도구는 셋업/비용 부담
문법·런타임 오류 디버깅 어려움	초보 이탈, 생산성 저하	검색/StackOverflow는 컨텍스트 불일치
실행 환경 불일치	“내 컴에선 되는데” 문제	로컬 설치·컨테이너 장벽
AI 도구 파편화	Chat / 리뷰 / 실행 앱 분리	컨텍스트 복사 반복, UX 단절
팀 리뷰에서 AI 활용 미흡	단순 코멘트 수준	AI 결과를 승인 워크플로에 연결 부족

2.2 기회

LLM API의 저지연·저비용화와 브라우저 기반 코드 샌드박스의 성숙으로, 교육·부트캠프·사이드 프로젝트·소규모 팀에서 “가벼운 AI 코드 리뷰 허브” 수요가 증가하고 있다. CodeReview AI는 이 구간에 풀스택 품질 루프(작성→분석→수정→실행→협업)를 제공한다.

3. 목표 사용자 및 페르소나

페르소나	니즈	주요 기능
학습자 / 부트캠프생	빠른 피드백, 오류 원인 이해	리뷰 점수, 오류 수정, 샘플, 챗봇
개인 개발자	혼자 작업 시 2차 검토	리뷰, 최적화, 실행, GitHub 로드
팀 리드 / 멘토	일관된 1차 스크리닝	협업 PR, AI 하이브리드 리뷰
소규모 스타트업 팀	가벼운 PR 프로세스	브랜치·PR·승인·머지 규칙

4. 제품 목표 / 비목표

4.1 In Scope (현재 프로토타입 + 웹 전환 1차)

- 다국어 코드 에디터(입력·샘플·줄번호·언어 선택/자동감지)
- AI 코드 리뷰(점수·등급·스타일/성능/안전/가독성·복잡도·버그·리팩토링 등)
- AI 오류 수정 및 수정본 에디터 적용
- 60점 미만 시 AI 자동 최적화 모달
- 코드 실행(Piston) + stdin + stdout/stderr 표시
- GitHub 파일 URL 불러오기
- 프로젝트 단위 챗봇(로컬 영속, 코드 컨텍스트 첨부)
- 협업 패널: 저장소·브랜치·PR·Diff·코멘트·승인·머지·AI 리뷰
- Groq API Key 사용자 입력(sessionStorage)

4.2 Out of Scope (1차 웹 구현에서 제외 권고)

- 실제 Git 서버/원격 push·pull (현재는 localStorage 시뮬레이션)
- 실시간 멀티유저 동시 편집(CRDT/WebSocket)
- SSO/OAuth 정식 계정 시스템(향후 로드맵)
- 유료 결제·구독 빌링
- 모바일 네이티브 앱
- 프로덕션급 샌드박스 격리(자체 실행 인프라)
- IDE 플러그인(VS Code 등)

5. 성공 지표 (KPI)

지표	정의	1차 목표
TTFR (Time to First Review)	키 입력 후 첫 리뷰 결과까지 시간	p50 ≤ 15초
리뷰 완료율	리뷰 시작 대비 결과 렌더 성공 비율	≥ 90%
수정 적용률	오류 수정 후 에디터 적용 비율	≥ 40%
실행 성공률	지원 언어 실행 요청 중 exit 0 비율	모니터링
AI 하이브리드 사용	PR 중 AI 리뷰 실행 비율	데모 시나리오 100%
JSON 파싱 성공률	LLM 응답 파싱 성공	≥ 95% (폴백 포함)

6. 시스템 아키텍처 개요

6.1 현재(As-Is) 아키텍처

브라우저 단일 페이지가 UI-상태·API 호출을 모두 담당한다. 서버 백엔드 없음. LLM·실행은 각각 Groq·Piston 공개 엔드포인트로 직접 호출한다.

[Browser SPA] → Groq API (Chat Completions)
[Browser SPA] → Piston API (Execute)
[Browser SPA] → GitHub raw.githubusercontent.com
[Browser SPA] ↔ sessionStorage / localStorage

6.2 목표(To-Be) 웹 아키텍처 권고

프론트엔드(SPA/SSR) + 백엔드 BFF(API Key 보호·프롬프트 관리) + 선택적 DB(사용자·협업 실데이터).

[Web Client] → [BFF / Backend] → Groq / Piston / GitHub
[Web Client] → [Auth] → [DB: users, projects, chats, repos, prs]

6.3 논리 모듈

모듈	책임	현재 위치
Editor Module	코드 입력, 샘플, 언어, 줄번호	메인 Script 1
Review Module	리뷰 요청·점수·섹션 렌더	doAnalyze / renderReview
Fix Module	오류 수정·적용	doFix / renderFix
Optimize Module	저점수 자동 최적화 모달	doOptimize
Execute Module	Piston 실행 패널	runCode
Chat Module	프로젝트/채팅/히스토리	sendChat + store
GitHub Module	원격 파일 로드	loadGH
Collab Module	저장소·PR·Diff·AI 리뷰	Script 2 IIFE
LLM Gateway	모델 폴백·JSON 파싱	callGroq / parseJSON

7. 사용자 여정 (User Journey)

7.1 핵심 여정: 첫 코드 리뷰

1. 서비스 진입 → Groq API Key 배너 확인
2. console.groq.com에서 키 발급 후 저장 (localStorage)
3. 에디터에 코드 입력 또는 샘플 칩 선택 / GitHub URL 로드
4. 언어 선택 또는 자동 감지
5. 「코드 리뷰」 클릭 → 로딩 스텝 표시
6. 우측 패널에 등급·총점·4축 카드·상세 섹션 표시
7. (선택) 60점 미만이면 자동 최적화 실행 → 모달에서 적용
8. (선택) 「오류 수정」 또는 「실행」으로 검증 루프

7.2 협업 여정: AI+사람 하이브리드 PR 리뷰

1. 헤더 「협업」 → 전체화면 협업 패널
2. 저장소 선택/생성 → 브랜치 생성
3. PR 생성(before/after 코드, 리뷰어 지정)
4. Diff에서 라인 코멘트 / 리뷰 탭 코멘트
5. AI 리뷰 실행 → 항목별 수락·무시·논의
6. 사람 최종 승인 또는 수정 요청
7. 승인 수 충족 시 Merge (규칙에 따라 소스 브랜치 삭제)

8. 정보 구조 및 화면 구성

8.1 메인 레이아웃

영역	구성 요소
Header	로고, 모델 배지, Ready 상태, 협업 토글
Key Banner	Groq API Key 입력/저장 (키 있으면 숨김)
Left Pane	Editor / GitHub / Chat 탭 + 하단 실행 패널
Right Pane	코드 리뷰 / 오류 수정 탭
Modals	최적화 결과, 프로젝트명, PR 생성, 저장소 생성
Collab Overlay	저장소 사이드바 + 개요/브랜치/PR/활동/AI 허브

8.2 지원 언어

Python, JavaScript, TypeScript, Java, C, C++, Go, Rust, SQL (에디터 선택 + detectLang 휴리스틱 + Piston 런타임 매핑).

9. 비기능 요구사항 (NFR)

구분	요구사항
성능	UI 상호작용 100ms 이내 피드백(로딩 표시 포함). LLM 호출은 비동기.

구분	요구사항
가용성	외부 API 장애 시 사용자 메시지 + 모델 폴백(429 시 순환).
보안	API Key는 sessionStorage. XSS 방지를 위해 사용자/LLM 출력 esc() 처리.
보안(향후)	서버 프록시로 Key 노출 제거, CSP, Rate limit, 입력 크기 제한.
호환성	최신 Chromium/Firefox/Safari. 뷰포트 720px 이하 2단→1단 스택.
접근성	버튼 레이블·키보드 Tab 들여쓰기. (향후: ARIA·포커스 트랩 강화)
로컬화	UI·프롬프트·응답 기본 한국어.
영속성	채팅/협업 localStorage. 키는 탭 세션 단위.
확장성	모듈 분리 가능한 단일 책임 함수 구조 → 프레임워크 이전 용이.

10. 제약사항 · 리스크 · 로드맵

10.1 제약사항

- Groq/Piston 쿼터·가용성에 의존
- 브라우저에서 Key 노출 위험 (As-Is)
- 협업 데이터는 기기/브라우저 로컬에 한정
- LLM 환각으로 잘못된 수정 코드 가능 → 사람 검토 필요
- Piston 공개 인스턴스 성능·보안 한계

10.2 리스크 및 완화

리스크	영향	완화
LLM JSON 파싱 실패	리뷰/수정 불가	다중 parse 폴백, json_object 모드, 재시도
Rate Limit (429)	기능 중단	모델 순환 폴백, 안내 메시지
Key 유출	과금·남용	sessionStorage, 향후 BFF 프록시
잘못된 수정 적용	코드 손상	confirm 다이얼로그, 복사 분리
localStorage 용량	데이터 손실	메시지 상한, 향후 서버 저장

10.3 로드맵 (제안)

Phase	내용	산출
P0 (현재)	단일 HTML 프로토타입 완성	codereview-ai.html
P1	문서화 + 프론트 모듈 분리 (본 문서)	PRD/기능/API Spec PDF
P2	웹 앱 스캐폴딩 (Vite+React 등) + BFF	배포 가능한 웹
P3	계정·서버 저장 채팅/협업	멀티 디바이스 동기화
P4	실 Git 연동 / 팀 워크스페이스	프로덕션 협업

PART B

기능명세서 (Functional Specification)

1. 기능 목록 총괄

ID	기능명	우선순위	모듈
F-01	API Key 등록/검증/복원	P0	Auth/Key
F-02	코드 에디터 (입력·줄번호·Tab)	P0	Editor
F-03	언어 선택 및 자동 감지	P0	Editor
F-04	샘플 코드 칩	P1	Editor
F-05	AI 코드 리뷰	P0	Review
F-06	점수 카드 상세 펼침	P1	Review
F-07	AI 오류 수정	P0	Fix
F-08	수정 코드 복사/에디터 적용	P0	Fix
F-09	AI 자동 최적화 (60점 미만)	P1	Optimize
F-10	코드 실행 (Piston)	P0	Execute
F-11	stdin / 실행 결과 패널	P1	Execute
F-12	GitHub 파일 로드	P1	GitHub
F-13	AI 챗봇 + 프로젝트/채팅 관리	P0	Chat
F-14	챗봇 코드블록 에디터 적용	P1	Chat
F-15	협업 저장소 CRUD	P1	Collab
F-16	브랜치 생성/삭제	P1	Collab
F-17	PR 생성/닫기/머지	P0	Collab
F-18	Diff + 인라인 코멘트/리액션	P1	Collab
F-19	PR 승인 및 머지 규칙	P0	Collab
F-20	PR AI 하이브리드 리뷰	P0	Collab/AI
F-21	활동 로그	P2	Collab
F-22	모델 풀백 및 상태 बै지	P1	LLM Gateway

우선순위: P0=필수, P1=중요, P2=있으면 좋음

2. 모듈별 상세 기능 명세

F-01 API Key 등록/검증/복원

목적: Groq API 사용을 위한 사용자 키를 안전하게 세션에 보관하고 인증 실패를 방지한다.

액터: 사용자

사전조건: 브라우저 접속

기본 흐름

1. 상단 키 배너에 gsk_ 로 시작하는 키 입력
2. 저장 클릭 또는 Enter
3. 형식 검증 후 sessionStorage에 저장
4. 배너 숨김 및 성공 메시지 표시

대안/예외 흐름

- gsk_ 미시작 → 알림 후 저장 거부
- 다른 탭에서 키 변경 시 500ms 폴링으로 동기화
- 키 없을 때 리뷰/수정/챗/AI 호출 → 배너 재표시 및 포커스

사후조건: GROQ_KEY 메모리 변수 및 sessionStorage 동기화

UI: #key-banner, #groq-key, #save-key-btn

비즈니스 규칙

- 키는 서버로 전송하지 않음(As-Is). 요청 시 Authorization Bearer로만 사용.

F-02~F-04 코드 에디터 · 언어 · 샘플

목적: 사용자가 분석 대상 코드를 입력·선택하고 언어 컨텍스트를 설정한다.

액터: 사용자

사전조건: 앱 로드 완료

기본 흐름

1. textarea에 코드 입력 → 줄번호 동기 갱신
2. Tab 키 시 스페이스 2칸 삽입
3. 입력 후 800ms 디바운스로 언어 자동 감지 및 select 갱신
4. 샘플 칩 클릭 시 해당 Python 샘플 로드

대안/예외 흐름

- 감지 실패 시 기존 언어 유지

사후조건: cta 값, lsel 값, 줄번호 DOM 갱신

UI: #cta, #lnums, #lsel, #chips-wrap

비즈니스 규칙

- 초기 샘플: buggy (보안/성능 이슈 데모 코드)
- 샘플 목록: split, bubble, fib, bsearch, linked, buggy, syntax

F-05~F-06 AI 코드 리뷰

목적: 코드 품질을 다측 평가하고 실행 가능한 개선 제안을 제공한다.

액터: 사용자, Groq LLM

사전조건: 유효 API Key, 비어 있지 않은 코드

기본 흐름

1. 「코드 리뷰」 클릭
2. 우측 리뷰 탭 전환, 로딩 스텝 표시

3. callGroq JSON 모드로 분석 요청
4. 응답 파싱 후 총점·등급·4축 카드·섹션 렌더
5. 카드 클릭 시 reason/problems/tips 펼침

대안/예외 흐름

- 401 → API 키 오류 메시지
- 429 → 다음 모델로 재시도
- 파싱 실패 → err-box

사후조건: 리뷰 결과 UI 표시. total<60 이면 최적화 버튼 노출

UI: #runbtn, #rp-review, #eval-banner, #score-cards, #rc

비즈니스 규칙

- 등급 매핑: $A \geq 80$, $B \geq 70$, $C \geq 60$, $D \geq 50$, else F (응답 grade 우선)
- 섹션: 복잡도, 스타일, 버그, 리팩토링, 알고리즘, 자료구조
- severity: error | warning | info | success

F-07~F-08 AI 오류 수정

목적: 문법·로직·런타임·보안·타입 오류를 탐지하고 수정된 전체 코드를 제공한다.

액터: 사용자, Groq LLM

사전조건: 유효 API Key, 코드 존재

기본 흐름

1. 「오류 수정」 클릭 → 우측 fix 탭
2. LLM에 오류 목록+fixed_full_code JSON 요청
3. 파싱 실패 시 fixed_full_code 필드 휴리스틱 추출
4. 오류 카드(Before/After) 및 전체 코드 테이블 렌더
5. 복사 또는 에디터 적용(confirm)

대안/예외 흐름

- 오류 0건 → 정상 축하 UI
- 실패 → err-box

사후조건: lastFixedCode 저장, fix-dot 표시(오류 있을 때)

UI: #fixbtn, #rp-fix, #fix-content

비즈니스 규칙

- 오류 타입 UI 매핑: SyntaxError, LogicError, RuntimeError, SecurityError, TypeError

F-09 AI 자동 최적화

목적: 저품질 코드에 대해 보안·성능·스타일 개선본을 생성한다.

액터: 사용자, Groq LLM

사전조건: 리뷰 총점 < 60 또는 사용자가 최적화 버튼 사용 가능 상태

기본 흐름

1. 최적화 버튼 → 모달 오픈 및 로딩
2. LLM 최적화 JSON 수신
3. 전/후 점수, changes, optimized_code 표시
4. 복사 또는 에디터 적용

대안/예외 흐름

- 파싱 실패 시 optimized_code 정규식 폴백

사후조건: lastOptCode 설정

UI: #opt-btn, #opt-overlay

비즈니스 규칙

- change.type: fix | improve | style

F-10~F-11 코드 실행

목적: 지원 언어 코드를 원격 런타임에서 실행하고 출력을 보여준다.

액터: 사용자, Piston API

사전조건: 코드 존재, 지원 언어

기본 흐름

1. 「실행」 클릭 (필요 시 언어 자동 감지 반영)
2. 실행 패널 open, running 상태
3. Piston /execute POST
4. compile/run stdout:stderr:exit code:소요시간 표시

대안/예외 흐름

- 미지원 언어 → 안내
- 네트워크 오류 → 친화적 메시지
- 복사/지우기/닫기 톨 제공

사후조건: 실행 결과 패널 갱신

UI: #execbtn, #exec-panel, #stdin-input, #exec-output

비즈니스 규칙

- 런타임 버전 고정 맵(LRT) 사용
- stdin은 input() 등에 전달

F-12 GitHub 파일 로드

목적: GitHub blob URL의 파일 내용을 에디터로 가져온다.

액터: 사용자, GitHub

사전조건: 공개 URL 또는 유효 토큰

기본 흐름

1. URL 입력 (선택 토큰)
2. github.com/.../blob/... → raw.githubusercontent.com 변환
3. fetch 후 에디터 채움

대안/예외 흐름

- HTTP 오류 → alert

사후조건: 에디터 탭 전환 및 코드 설정

UI: #p-github, #ghurl, #ghtok

비즈니스 규칙

- 토큰은 Authorization: token 헤더로만 사용

F-13~F-14 AI 챗봇

목적: 코드 구현·설명·최적화를 대화형으로 지원하고 히스토리를 영속화한다.

액터: 사용자, Groq LLM

사전조건: 유효 API Key

기본 흐름

1. 채팅 탭 → 메시지 입력/제안 칩
2. 현재 에디터 코드를 컨텍스트로 첨부
3. history 최대 20턴 유지
4. 응답 마크다운 간단 렌더, 코드블록 추출 시 적용 버튼
5. localStorage cr_chats_v3 저장

대안/예외 흐름

- 첫 메시지 시 채팅 제목 자동 생성(22자)
- 프로젝트 폴더로 채팅 그룹화

사후조건: 채팅 메시지·history 저장

UI: #p-chat, #chat-msgs, #chat-input

비즈니스 규칙

- store: projects[], chats[], activeChatId
- chat: id, title, projectId, history[], messages[], timestamps

F-15~F-16 협업 저장소·브랜치

목적: 로컬 시뮬레이션 저장소와 브랜치를 관리한다.

액터: 사용자 (ME='me')

사전조건: 협업 패널 오픈

기본 흐름

1. 새 저장소: 이름/설명/멤버 → main 브랜치 자동 생성
2. 브랜치 생성/삭제(default 제외)
3. 검색으로 저장소 필터

사후조건: localStorage codereview_collab_v2 갱신 + 활동 로그

UI: #collab-panel, #cb-repo-list

비즈니스 규칙

- 시드 데이터: demo-project + feature/ai-review PR

F-17~F-19 PR · Diff · 승인 · 머지

목적: 변경 전후 코드 기반 PR 워크플로를 제공한다.

액터: 작성자, 리뷰어

사전조건: 저장소·브랜치 존재

기본 흐름

1. PR 생성: 제목, src/tgt, 설명, 리뷰어, before/after
2. Diff(LCS) unified + side-by-side
3. 라인 클릭 인라인 코멘트, 리액션 토글
4. 리뷰어 승인 시 approvals에 ME 추가
5. approvals.length ≥ minApprovals 이면 Merge 가능
6. Merge 시 status=merged, 옵션에 따라 소스 브랜치 삭제

대안/예외 흐름

- Close → status=closed

사후조건: PR 상태·로그·브랜치 갱신

UI: PR 모달, PR 상세 서브탭

비즈니스 규칙

- mergeRules: requireApproval, minApprovals, requireCI, deleteBranch
- requireCI는 UI 토글만 존재(실행 연동 없음, As-Is)
- status: open | merged | closed

F-20 PR AI 하이브리드 리뷰

목적: AI가 PR diff를 1차 분석하고 사람이 항목 단위로 판단·최종 승인한다.

액터: 리뷰어, Groq LLM

사전조건: open PR, API Key

기본 흐름

1. AI 분석 시작 → before/after 기반 JSON 리뷰

2. score 및 category items 표시
3. 항목별 수락/무시/논의(hd 필드)
4. 사람 노트와 함께 승인 / 수정 요청 / AI 무시 강제 승인

대안/예외 흐름

- 모델 3단 폴백
- 키 없으면 alert

사후조건: pr.aiReviews() push, activity/log 기록

UI: PR 상세 AI 탭, AI 허브 탭

비즈니스 규칙

- category: bug|performance|style|security|test
- severity: error|warning|info|suggestion
- human decision: accept|reject|discuss

F-22 모델 폴백 및 상태 बै지

목적: Rate limit-장애 시 가용 모델로 전환하고 UI에 상태를 표시한다.

액터: 시스템

사전조건: LLM 호출

기본 흐름

1. 기본 모델부터 요청
2. HTTP 429 시 modelIdx 증가 후 1초 대기 재시도
3. 모든 모델 실패 시 에러
4. 헤더 모델 बै지-Ready/Analyzing/Fixing 상태 갱신

사후조건: modelIdx 갱신

UI: #model-badge, #sbadge

비즈니스 규칙

- 모델 목록: llama-3.3-70b-versatile, llama-3.1-8b-instant, gemma2-9b-it, mixtral-8x7b-32768
- temperature 기본 0.1, max_tokens 리뷰 8192 / AI PR 3000

3. 데이터 모델 (클라이언트)

3.1 채팅 스토어 (localStorage: cr_chats_v3)

```
Store { projects: Project[], chats: Chat[], activeChatId: string|null }
Project { id, name, color, chatIds[], collapsed }
Chat { id, title, projectId, history: {role,content}[], messages: {role,html,time,code}[], createdAt, updatedAt }
```

3.2 협업 스토어 (localStorage: codereview_collab_v2)

```
CollabState { repos[], branches[], prs[], log[] }
Repo { id, name, desc, owner, members[], color, emoji, mergeRules, createdAt }
Branch { id, repold, name, isDefault, author, commits, lastCommit, updatedAt }
PR { id, repold, title, desc, src, tgt, author, reviewers[], status, approvals[], before, after, comments[], activity[], aiReviews[], createdAt }
Comment { id, lineNum, content, author, reactions, createdAt }
AIReview { id, model, score, bugScore, perfScore, styleScore, secScore, testScore, summary, items[], humanApproved, createdAt }
```

3.3 세션 (sessionStorage)

groq_key: string // gsk_... 형식의 Groq API Key

4. 권한 · 상태 전이

4.1 PR 상태 전이

open --(merge, 승인조건 충족)--> merged
open --(close)--> closed
merged/closed 는 단말 상태 (As-Is 재오픈 없음)

4.2 권한 (As-Is 단순 모델)

행위	조건
저장소 생성	누구나 (로컬 ME)
PR 승인	reviewers에 ME 포함 & 아직 미승인
Merge 버튼	status=open & approvals ≥ minApprovals
브랜치 삭제	isDefault=false
강제 승인	AI 탭에서 사용자 confirm

5. 예외 및 오류 처리

상황	사용자 메시지/동작
키 없음	키 배너 표시, needKey() true
키 형식 오류	gsk_ 시작 안내 alert
401 Unauthorized	API 키 인증 실패 안내
429 Rate Limit	다음 모델 자동 전환 재시도
JSON 파싱 실패	err-box 또는 필드 휴리스틱 폴백
Piston 네트워크 오류	인터넷 확인 안내
GitHub fetch 실패	HTTP status alert
빈 코드 실행/리뷰	입력 요청 alert
에디터 적용	confirm 후 교체, 1.5s 그린 아웃라인

PART C

API명세서 (API Specification)

1. 외부 API 연동 개요

API	Base URL	용도	인증
Groq Chat	https://api.groq.com/openai/v1	리뷰·수정·챗·PR AI	Bearer groq_key
Piston	https://emkc.org/api/v2/piston	코드 실행	없음
GitHub Raw	https://raw.githubusercontent.com	파일 내용	Optional token

본 절은 As-Is 클라이언트 직접 호출 규약을 정의한다. To-Be에서는 동일 계약을 BFF가 중계한다.

2. Groq Chat Completions

2.1 엔드포인트

POST /openai/v1/chat/completions

2.2 Request Headers

Header	값	필수
Authorization	Bearer {GROQ_KEY}	Y
Content-Type	application/json	Y

2.3 Request Body

필드	타입	설명
model	string	모델 ID (폴백 목록 중 하나)
messages	array	[{role, content}] OpenAI 호환
temperature	number	기본 0.1
max_tokens	number	리뷰 8192, 챗 8192, PR AI 3000
response_format	object	선택. {type:"json_object"} 리뷰 시

2.4 사용 모델

순서	model id	표시 라벨
1	llama-3.3-70b-versatile	Llama 3.3 70B
2	llama-3.1-8b-instant	Llama 3.1 8B
3	gemma2-9b-it	Gemma 2 9B
4	mixtral-8x7b-32768	Mixtral 8x7B

PR AI 탭 전용 폴백 목록에는 Mixtral 미포함 (1~3만 사용).

2.5 Response (성공)

OpenAI 호환 JSON. 클라이언트는 `choices[0].message.content` 문자열만 사용한다.

```
{ "choices": [ { "message": { "role": "assistant", "content": "..." } } ] }
```

2.6 Error Handling

HTTP	클라이언트 동작
401	Error: API 키 인증 실패
429	modelidx++, 1s wait, 다음 모델 재시도 (최대 모델 수)
기타 4xx/5xx	body.error.message 또는 HTTP {status} throw
네트워크	fetch rejection → UI err-box

2.7 기능별 messages 계약

기능	system	user 요약	jsonMode
코드 리뷰	수석 엔지니어. 순수 JSON	언어+코드 + scores/sections 스키마	true
오류 수정	오류 수정 전문가. JSON	코드 + errors/fixed_full_code 스키마	false
최적화	수석 엔지니어. JSON	원본 코드 + optimized_code 스키마	false
챗봇	전문 코드 AI. 한국어	history + 에디터 코드 컨텍스트	false
PR AI	코드 리뷰 AI. JSON	PR 제목 + before/after	false

3. Piston Code Execution

3.1 엔드포인트

POST <https://emkc.org/api/v2/piston/execute>

3.2 Request Body

```
{ "language": "python", "version": "3.10.0", "files": [ { "name": "main.py", "content": "..." } ], "stdin": "optional" }
```

필드	타입	설명
language	string	Piston 언어 식별자
version	string	고정 런타임 버전
files	array	[{name, content}] 실행 파일
stdin	string	선택. 표준 입력

3.3 언어 런타임 매핑 (LRT)

UI 언어	Piston language	version	filename
python	python	3.10.0	main.py
javascript	javascript	18.15.0	main.js
typescript	typescript	5.0.3	main.ts

UI 언어	Piston language	version	filename
java	java	15.0.2	Main.java
c	c	10.2.0	main.c
cpp	c++	10.2.0	main.cpp
go	go	1.16.2	main.go
rust	rust	1.50.0	main.rs
sql	sqlite3	3.36.0	main.sql

3.4 Response (사용 필드)

경로	설명
run.stdout	표준 출력
run.stderr	런타임 에러
run.code	프로세스 종료 코드 (0=성공)
compile.stdout / compile.stderr	컴파일 단계 출력(C/Java 등)

4. GitHub Raw Content

4.1 URL 변환 규칙

입력: `https://github.com/{user}/{repo}/blob/{branch}/{path}`
변환: `https://raw.githubusercontent.com/{user}/{repo}/{branch}/{path}`
(구현: host 치환 + `/blob/'` → `'/'`)

4.2 Request

항목	내용
Method	GET
Headers	선택 Authorization: token {ghtok}
Response	text/plain 파일 본문

5. 프롬프트 계약 (JSON Schema — 논리)

LLM이 반환해야 하는 JSON 형태. 런타임 JSON Schema 검증은 없으나 클라이언트가 이 구조를 기대한다.

5.1 코드 리뷰 응답

```
{
  "scores": { "total", "style", "performance", "safety", "readability", "grade", "summary" },
  "score_details": {
    "style|performance|safety|readability": {
      "reason", "problems": [], "tips": [{ "title", "desc", "code" }]
    },
    "style": [{ "severity", "title", "description", "suggestion" }],
    "complexity": { "time", "space", "rating", "explanation", "items": [] },
    "bugs": [], "refactoring": [], "algorithms": [], "datastructures": []
  }
}
```

5.2 오류 수정 응답

```
{
  "has_errors": true,
  "error_count": 2,
  "summary": "...",
  "errors": [{
    "type": "SyntaxError|LogicError|RuntimeError|SecurityError|TypeError",
    "line": 3, "original": "...", "problem": "...", "fix": "...", "fixed_code": "..."
  }],
  "fixed_full_code": "..."
}
```

5.3 최적화 응답

```
{
  "summary", "score_before", "score_after",
  "changes": [{ "type": "fix|improve|style", "title", "detail" }],
  "optimized_code": "..."
}
```

5.4 PR AI 리뷰 응답

```
{
  "score", "bugScore", "perfScore", "styleScore", "secScore", "testScore",
```

```

"summary",
"items": [{
  "id", "category": "bug|performance|style|security|test",
  "severity": "error|warning|info|suggestion",
  "title", "description", "line", "suggestion"
}] // 2~6개 권장
}

```

5.5 JSON 파싱 폴백 순서 (parseJSON)

1. JSON.parse(raw)
2. ``json 펜스 제거 후 parse
3. 일반 `` 펜스 제거 후 parse
4. 첫 '{' ~ 마지막 '}' 슬라이스 후 parse
5. 기능별 추가: fixed_full_code / optimized_code 필드 정규식 추출

6. 향후 백엔드 API 설계 초안 (To-Be)

웹 정식 구현 시 클라이언트에 Key를 두지 않도록 BFF를 둔다. 아래는 권장 REST 초안이다.

6.1 공통

항목	규약
Base Path	/api/v1
Auth	Authorization: Bearer {access_token} (사용자 세션)
Content-Type	application/json
에러 포맷	{ "error": { "code", "message", "details?" } }
Rate Limit	X-RateLimit-* 헤더, 429 + Retry-After

6.2 AI 프록시

Method	Path	설명
POST	/ai/review	코드 리뷰. body: {language, code} → ReviewJSON
POST	/ai/fix	오류 수정. body: {language, code} → FixJSON
POST	/ai/optimize	최적화. body: {language, code} → OptJSON
POST	/ai/chat	챗 완성. body: {messages, language?, code?} → text
POST	/ai/pr-review	PR AI. body: {title, before, after} → AIReviewJSON

6.3 실행 프록시

Method	Path	설명
POST	/execute	body: {language, code, stdin?} → {stdout, stderr, code, time}
GET	/execute/runtimes	지원 언어·버전 목록

6.4 채팅·프로젝트 (서버 영속)

Method	Path	설명
GET	/projects	프로젝트 목록
POST	/projects	프로젝트 생성
DELETE	/projects/{id}	프로젝트 삭제
GET	/chats	채팅 목록 (?projectId)
POST	/chats	채팅 생성
GET	/chats/{id}	메시지 포함 상세
POST	/chats/{id}/messages	메시지 전송(서버가 LLM 호출 가능)
DELETE	/chats/{id}	채팅 삭제

6.5 협업 (서버 영속)

Method	Path	설명
GET/POST	/repos	저장소 목록/생성
GET/POST	/repos/{id}/branches	브랜치
GET/POST	/repos/{id}/prs	PR 목록/생성
GET	/prs/{id}	PR 상세
POST	/prs/{id}/comments	코멘트
POST	/prs/{id}/approve	승인
POST	/prs/{id}/merge	머지
POST	/prs/{id}/close	닫기
POST	/prs/{id}/ai-reviews	AI 리뷰 실행·저장
PATCH	/ai-reviews/{id}/items/{itemId}	수락/무시/논의
GET	/repos/{id}/activity	활동 로그

6.6 내부 시퀀스 예: 코드 리뷰

```
Client → POST /api/v1/ai/review {language, code}
BFF → validate auth, size limit, inject system prompt
BFF → Groq chat/completions (json_object, model fallback)
BFF → parseJSON + optional schema validate
BFF → Client 200 ReviewJSON
```

부록. 용어집 · 참고

A. 용어집

용어	설명
PRD	Product Requirements Document. 제품 요구사항 정의서
BFF	Backend for Frontend. 프론트 전용 API 게이트웨이
Piston	오픈소스 원격 코드 실행 엔진 (emkc 호스팅 인스턴스 사용)
Groq	고속 LLM 추론 API 제공 플랫폼
하이브리드 리뷰	AI 1차 분석 + 사람 최종 판단 워크플로
LCS Diff	Longest Common Subsequence 기반 라인 diff
jsonMode	Groq response_format=json_object 요청
ME	협업 시뮬레이터의 현재 사용자 식별자 ('me')

B. 현재 소스 기준 파일

codereview-ai.html — CSS + 메인 UI + Script1 (리뷰/수정/실행/챗) + 협업 UI + Script2

C. 참고 링크

- Groq Console Keys: <https://console.groq.com/keys>
- Groq OpenAI-compatible API: <https://api.groq.com/openai/v1/chat/completions>
- Piston API: <https://github.com/engineer-man/piston>
- Piston Execute Host: <https://emkc.org/api/v2/piston/execute>

문서 끝 — CodeReview AI Spec v1.0 · 2026-07-30 · 본 문서는 웹 구현의 기준 산출물이며, 구현 변경 시 버전을 갱신한다.